

Data Engineering Framework

[Link - Data Engineering 101](#)

Pre-Requisite: Understanding Business Needs and Goals

> Meet with stakeholders to understand Business needs and goals. Example: aggregate raw data into business ready clean data that is updated automatically.

> Document the success criteria during stakeholder meetings. Not just what the business needs, but how they will know the pipeline is working correctly. This becomes the basis for your data quality assertions later and prevents scope creep during build.

Progression Documentation Tools

- > In a business setting this would be Jira or Confluence.
- > In a personal setting this could be Notion.

Data Architecture

-> Based on business needs and goals, determine data management methods: **Data Architectures and Data Modeling Approaches**

You choose the architecture first based on data volume, access patterns, and team structure. You choose the modeling approach second based on the architecture chosen and the target audience. A Data Warehouse almost always pairs with Kimball or Inmon. A Lakehouse pairs well with Medallion. Data Vault is typically reserved for highly regulated industries requiring full auditability.

1. [Data Architecture](#):
 - a. [Data Warehouse](#)
 - b. [Data Lake](#)
 - c. [Data Lakehouse](#)
 - d. [Data Mesh](#)
2. [Data Models](#):
 - a. [Inmon](#)
 - b. [Kimball](#)
 - c. [Data Vault](#)
 - d. [Medallion](#) - [Extra Link](#)

-> **Separation of Concern**: For each Data Model Layer, Data Engineer must;

1. Define the Layer
 - a. Document who owns that layer. In a team setting, ownership boundaries prevent engineers from making changes to layers they do not understand.
2. Determine Objective of Layer
3. Determine Object type of Layer
 - a. [Tables](#)
 - b. [Views](#)
4. [Determine Load Method](#) (if any) - add **scheduling frequency** here. Full load implies a different schedule than incremental. The frequency decision affects orchestration design downstream.

- a. [Batch vs Streaming Processing](#)
 - b. Full Load vs Incremental
 - c. Truncate and Insert types
- 5. Determine Data Transformations (if any) - add **data type casting** as a transformation type. One of the most common sources of silent pipeline failures is a column arriving as a string that downstream logic expects as an integer.
 - a. Cleansing
 - b. [Standardizations](#)
 - c. [Normalization](#)
 - d. [Derived Columns](#)
 - e. [Data Enrichment](#)
 - f. Data Integrations
 - g. Aggregations
 - h. Business Logic
- 6. Data Modeling (if any)
 - a. [Star Schema Model](#)
 - b. [Snowflake Schema Model](#)
- 7. [Determine Target Audience of the Data](#) - this decision should also influence your data model choice. A Data Scientist consuming the data needs different structures than a BI Analyst. This feeds back into whether you build a Star Schema or a flatter, wider table.
 - a. [Data Engineer](#)
 - b. Data Analyst (BI Analysts or Insight Analysts)
 - c. Data Scientist (AI and MLL)

-> Visualize/Document the Data Architecture presented above ([Draw.io](#) as a tool) - the architecture diagram should be versioned alongside the code in Git. An outdated diagram is worse than no diagram because it actively misleads anyone reading it.

-> Determine all necessary Data Tools - The IDE and local database system are local concerns. The cloud storage and orchestration tool are production concerns.

- 1. Cloud Based Storage: AWS, Azure, GCP
- 2. Data Infratechnologies: Coding IDE, Database Systems, Orchestration Tool, Git
- 3. Recalling Data Engineering Concepts
 - a. Data Immutability
 - b. Auditability
 - c. Slowly Changing Dimensions
 - d. Pipeline Dependencies
 - e. ACID
 - f. CI/CD
 - g. Idempotency
 - h. Write Strategies
 - i. Containerization and Environment Constraints
 - j. Local-Production Environment Best Practices
 - k. Scheduling, Retires, and Failure Handling Pipelines
 - l. Pipeline Monitoring Backfilling, and Observability

- m. **Data Lineage** — knowing where data came from and what touched it before it reached a given layer
- n. **Schema Evolution** — how the pipeline handles upstream source schema changes without breaking downstream layers

Project Initiation

> Once everything has been established above (Business Needs, Data Architecture, Data Engineering Tools), the project and pipeline development can begin.

> Establish a convention for environment prefixes as well. Tables in development should be distinguishable from tables in production at a glance. A common pattern is `dev_`, `stg_`, `prod_` prefixes on schema names.

-> Establish Naming Convention for Project, Tables, and Columns

1. Avoid using reserved words for a given programming language (e.g don't use `crm_create`, `create` is a SQL command)
2. Naming Standard can be `<sourcesystem>_<entity>`
 - a. Sourcesystem: Name of the source system (S3, name of folder, etc)
 - b. Entity: Exact table name from source system

-> Start Git Repository

1. Start Git Structure (Create w/ generalized placeholder names)
 - a. Dataset folder (Houses datasets, if any)
 - b. Document folder (Houses all thinking process documents)
 - c. Script folder (Houses all programming language scripts)
 - d. Test folder (Houses assertion scripts - test for data quality and validation)
 - e. **Environments folder** or `.env.example` file at the root level for credential templates
 - f. **Migrations folder** for database schema change scripts — this becomes critical when your schema evolves and you need to track what changed and when
2. Adjust Read Me
 - a. Used to document full project
 - i. Goal of project
 - ii. Tools used
 - iii. Source and Drain systems
 - iv. Document thinking process of Project
 - v. Etc

-> Create Databases and Schema for the different Layers

-> Push Initiate Database Script from above into Git Repository

1. Make sure to include header details for future reference

-> The 4 Cycles of a Layer

1. Understanding Upstream or Source Systems (Analysis Stage of Layer)
 - a. Understand; this is done by speaking with Data Source System Owners
 - i. Data Ownership

- ii. Business Process
 - iii. System and Data Documentations
 - iv. DataModel and Catalog
- b. Asking;
 - i. How is the Data Stored? (e.g SQL Server, Oracle, AWS, Azure, etc)
 - ii. Integration Capabilities? (e.g API, Kafka, File Extract, Direct DB, etc)
- c. Extract and Loading Planning
 - i. Data Profiling the Source Data - understand its volume, null rates, duplicate rates, and update frequency
 - ii. Choosing between Full Loads or Incremental
 - iii. Data Scope and Historical needs (Lookback range of data)
 - iv. What is the exact size of the extracts?
 - v. Volume Limitations
 - vi. Authentication and Authorization requirements? (e.g tokens, ssh keys, IP Whitelisting, etc)
 - vii. **Note: Avoid impacting performance of source system**
- 2. Data Ingestion + Transformation (Coding Stage of Layer)
 - a. Develop Table and Conditions (Handle 'existing table' errors, etc)
 - b. Determine Ingestion Type (**Write Strategies**)
 - c. Analyze ingested to validate expected schema
 - d. Save executable stores procedures to easily run script queries
 - e. Idempotency Validation - confirm that running the ingestion script twice produces the same result as running it once
- 3. Data Completeness and Schema Checks (Validation Stage of Layer)
 - a. Make Sure that all Assertions are meet
 - i. Standard Assertion: Row Count Reconciliation - Confirm that the number of records ingested matches what the source system reported
 - b. Create Logging Procedures to track potential bottlenecks against future SLA violations.
 - c. Documented header with potential error flags of script
- 4. Data Documentation Versioning in Git (Docs & Version Stage of Layer)
 - a. Changelog Entry - everytime a push is committed, one line entry in the changelog documents what changed and why
- 5. Same as before, push everything associated with the Layer to Git before moving on to the next Layer.

-> Data Modeling (Specific to Data Model Layer either Gold Layers or Data Marts)

1. Conceptual Data Model - Determine the entities/tables needed and their relationships to each other.
2. Logical Data Model - Specify the different columns associated with tables above. Determine relationship between tables (Primary and Foreign Keys)
3. Physical Data Model - Specify datatypes of each column above and constraints. Determine length of values and other database value assertions

- Once all three have been documented/visualized, know the query patterns to determine **Star or Snowflake Schema Models** - High join complexity favors Star. Storage efficiency and normalization requirements favor Snowflake (Remember [Facts vs. Dimension Theory](#))
- For these tables, also apply Data Transformations as needed!

-> Develop a Data Catalog (Completion of Data Modeling)

1. Database Project Dictionary and Vocabulary
 - a. Overview
 - b. Table Names and Purpose
 - c. Columns Names and Values
 - d. Column Data Types
 - e. Description of Column Values
 - i. Add examples and Formatting descriptions
2. Data Lineage Documentation - For each table, document not just what the columns mean but where the data came from and what transformations it passed through to get there

-> For a personal project, this would be the end, however, we want to turn this into a Data Pipeline that we can manage and control:

1. Data Orchestration - Regulates scheduled automatic runs for the stores above GIT scripts; sits between the 4 Cycles section and the Dashboard section
 - a. DAG design for each layer's scripts — ingestion, transformation, validation, in order
 - b. Scheduling frequency per layer (e.g. Bronze may run hourly, Gold may run daily)
 - c. Retry strategy per task — defined based on the failure characteristics of each source system
 - d. Failure callbacks and alerting — who gets notified and how
 - e. SLA definition per DAG — what does on-time completion mean for this pipeline
 - f. Backfill strategy — how historical data is handled if the pipeline is new or recovers from failure
2. Production Environment - Client facing scripts and interfaces; sits after orchestration and before dashboard development
 - a. Containerization: Dockerfile and Docker Compose definition for all pipeline services
 - b. Environment variable management: `.env` structure for local and production
 - c. CI/CD pipeline: how code moves from Git to production automatically after passing tests
 - d. Production monitoring: log aggregation, SLA tracking, and alerting configuration
 - e. Blue-green deployment strategy: how pipeline updates are deployed without disruption
3. Developing Dashboards for Business Insights - Business Insights Analyst responsibilities